MONDAY, JULY 19

—

STUDYING SCAN.PY:

THE CODE USES SURYA INFERENCE MANAGER & RECOGNITION PREDICTOR. WHAT ARE THESE?

→ INFERENCE MANAGER:

- RESPONSIBLE FOR LOADING THE MODEL, MANAGING MEMORY, THE "ENGINE". IF I SWITCH TO GOOGLE COLAB, I COULD USE AN NVIDIA GPU INSTEAD OF LLAMA.CPP. SPEAKING OF WHICH, LET ME LOOK INTO GPU VS CPU.

→ RECOGNITION PREDICTOR:

- THIS IS ACTUALLY ONE OF THREE "MODES" SURYA OCR IS CAPABLE OF. APPARENTLY IT USED TO BE MORE DISPERSED AND SURYA 2 LETS YOU USE ONE INFERENCE MANAGER FOR ALL OF THESE MODALITIES. RECOGNITION PREDICTOR FOCUSES ON TEXT, THE OTHERS CAN ACTUALLY WORK WITH LAYOUTS/ STRUCTURE & TABLES. CAN RETURN TEXT AS HTML LIKE I DID.

- WHAT'S THE DIFFERENCE BETWEEN A GPU AND CPU?
- WHAT IS LATENCY VS THROUGHPUT?
- INTEGRATED CIRCUITS? TRANSISTORS?

WHILE BOTH CPUS AND GPUS HAVE MULTIPLE CORES...
WHAT IS THAT? A CORE IS ESSENTIALLY A MINI-COMPUTER, LIKE AN IBM SYS 360, IT HAS AN ARITHMETIC LOGIC UNIT, CONTROL UNIT, MEMORY/CACHE, ETC. SO GPUS HAVE MORE CORES, AND WHILE MADE FOR GRAPHICS INITIALLY, THEY'VE BEEN USED IN AI/ML WORKFLOWS. AND:

LATENCY = TIME TO DO TASK | THROUGHPUT = TASK LOAD (TASKS / SEC)

NOW. INSIDE THE FUNCTION, THE FIRST TASK IS TO READ IN ALL FILES IN THE DIR. I DUMPED THE PDF'S INTO. OS.PATH.JOIN() CREATES THE FULL DIRECTORY NAME SO THEY CAN BE FOUND AS THE CODE PROCESSES EACH PDF. OS.LISTDIR() READS THE DIRECTORY, AND IN THIS CASE IT'S WRAPPED IN SORTED() TO HAVE IT IN NUMERIC ORDER IN LIST COMPREHENSION.

NEXT IS ONE OF FOUR FOR LOOPS! THIS ONE LOOPS THROUGH EACH OF THE FILES I PULLED OUT OF MY DIRECTORY. THE NEXT LOOP GOES ONE LEVEL LOWER INTO PAGES WITHIN PDF'S.

FILES IN DIR (LIST) > ITERATE CH. BY CH. > PAGES INSIDE EACH CH.

GEMINI ALSO THOUGHT IT WAS WORTH INITIALIZING A VAR FOR GRABBING THE CH. NAME BEFORE THE FILE EXTENSION.

FOR EACH CHAPTER, FITZ OPENS THE FILE AS "DOC", AND USES ANOTHER UNHOLY FOR LOOP USING ENUMERATE(DOC). WHAT IS ENUMERATE?

SO ENUMERATE COMES IN HANDY WHEN YOU WANT TO TRACK AN INDEX. THIS RETURNS A TUPLE LIKE (INDEX, VALUE), AND THIS WAY YOU DON'T NEED AN ITERABLE INDEX LIKE FOR i=0... IN THIS CASE, PAGE_NUM = INDEX AND PAGE OF TYPE PYMPDF.DOC. THE INDEX IN THIS CASE IS USELESS, ONLY USED FOR PRINT STATEMENTS.

NOW FOR THE FUN PART: WHAT THE HELL IS PYMU / PIL / CV2, ETC. I WILL JUST FOCUS ON THIS CODE FOR NOW, ACTUALLY. SO

- PIX = PAGE.GET_PIXMAP(DPI=150)

PAGE ← PYMUPDF PAGE/ PYMUPDF DOC

GET_PIXMAP ← GENERATES A PIXMAP, WHICH IS A "PLANE RECTANGULAR SET OF PIXELS."

PIXMAP(DEVICERGB, (x, x, x, x), x)
: n

TRANSPARENCY "RGBA" → α

- IMG = IMAGE.FROMBYTES ("RGB", [PIX.WIDTH, PIX.HEIGHT], PIX.SAMPLES)

THIS TAKES THE PIXMAP DATA, LIKE (0, 0, 1319, 2084), AND READS IT LIKE (x_0, y_0, x_1, y_1). WIDTH = $x_1 - x_0 = 1319$, HEIGHT IS $y_1 - y_0 = 2084$. THIS IS DONE AUTOMATICALLY BY PYMUPDF'S PIX.WIDTH AND PIX.HEIGHT. APPARENTLY THIS IS A TUPLE, DESPITE THE LIST BRACKETS, BUT IS HANDLED BECAUSE OF PYTHON'S DUCK TYPING (IF IT WALKS LIKE A DUCK... MUST BE A DUCK). PIX.SAMPLES TELLS ~~PIL~~ PIL THE SIZE OF THE BYTE ARRAY, 3 (RGB) x 1319 x 2084 = 8.2 MIL BYTES, THEN FILLS IN THE ARRAY.

SURYA OCR: RESULTS = REC([IMG])

THE RECOGNITION PREDICTOR TAKES THE "IMG" AS AN ARGUMENT. HERE'S A REALLY COOL REALIZATION I HAD — IMG IS IN BRACKETS. SO SURYA'S RECOGNITION PREDICTOR IS DESIGNED TO TAKE LISTS. ALSO, WHEN I READ THE DOCUMENTATION, SURYA AUTOMATICALLY IMPLEMENTS PARALLEL PROCESSING. SO BY DUMPING EACH PDF INTO A FOR LOOP, THIS DEFEATS THE PURPOSE. I CAN ACTUALLY SEND THE WHOLE LIST OF PAGES IN AT ONCE, AND PARALLELIZE THE ENTIRE CHAPTER SO THEY'RE PROCESSED SIMULTANEOUSLY!

F.L.
-1

→ RESULTS = REC(CHAPTER_IMAGES)

"RESULTS" IS NESTED CONTENT: PAGES > BLOCKS > CONTENT

- PAGES = EXACT LIST I PASSED IN, RESULTS[0] = 1ST PAGE
- BLOCKS = CHUNKS, TEXT UNDER SUBHEADINGS, ETC.
- CONTENT = WHAT'S IN THE BLOCK?
  - "TEXT": "TEXT"
  - "HTML": <H2></H2>
  - "BBOX": WHERE ON PAGE
  - "POLYGON": SHAPE
  - + "CONFIDENCE": HOW CERTAIN?

. FOR BLOCK IN RESULTS[0].BLOCKS:

    MARKDOWN_CONTENT += MD(BLOCK.HTML) + "\n\n"

SO THIS CODE IS INTERESTED IN THE HTML IN THE BLOCK. USING MARKDOWNIFY, IT'S CONVERTED TO MARKDOWN AND APPENDED TO THAT LONGFORM STRING. ALL OF THIS IS WRITTEN TO THE NEW MARKDOWN FILE THAT I'LL TURN INTO A WEBPAGE.